IFA - Höhere Fachschule der Digitalen Wirtschaft

Bogenschützenstrasse 9A

CH-3008 Bern

PROJEKTARBEIT - DOKUMENTATION

Online-Plattform zum Verleih von E-Scootern

5. Semester (Wintersemester 2025/2026)

Verfasser: Andreas Geiger

Studiengang: Dipl. Informatiker/-in HF, Applikationsentwicklung/ Application-Security

Modul: DBWE

Dozent: Christian Herren

Abgabetermin (letztmöglicher): 20.03.2026

Inhaltsverzeichnis

1. Ausgangslage 3
2. Management Summary 3
3. Zielsetzung 4
4. Funktionale Anforderungen 5
5. Nicht-funktionale Anforderungen 6
6. Lösungsansatz: Wahl der Versionskontrollplattform 7
7. Abgrenzungen 8
8. Datenbankmodell (ERD) 9
9. Architektur (Model-View-Controller) 10
10. Prozess / Workflow 11
11. API-Konzept 12
12. Bereitstellung der Anwendung (Deployment) 13
13. Realisierung 15
12.1 Technischer Aufbau und Grundlagen 15
12.2 Implementierung der Funktionalitäten 18
14. Testprotokoll 22
15. Fazit 23
16. Reflexion 24
17. Quellenverzeichnis 25
18. Abkürzungsverzeichnis 25
19. Glossar 25

1. Ausgangslage

Mit dem Wachstum vieler Städte und dem steigenden Wunsch nach flexibler und umweltfreundlicher Mobilität rücken alternative Verkehrsmittel wie E-Scooter immer stärker in den Fokus. Gerade in urbanen Gebieten führt der klassische Individualverkehr zunehmend zu Staus und höheren Emissionen. Um dem entgegenzuwirken, plant die Stadtverwaltung den Aufbau einer digitalen Plattform, die die Nutzung von E-Scootern einfacher, effizienter und komfortabler macht.

Diese Plattform soll sowohl für private Nutzerinnen und Nutzer als auch für kommerzielle Betreiber attraktiv sein. Verleihanbieter sollen ihre Scooter registrieren, verwalten und für die Nutzung freigeben können. Gleichzeitig soll es Nutzerinnen und Nutzern möglich sein, verfügbare Scooter schnell zu finden, zu reservieren, zu verwenden und am Ende unkompliziert abzurechnen.

Die Umsetzung erfolgt als Webapplikation, die über den Browser erreichbar ist und zusätzlich eine Schnittstelle für externe Anwendungen bereitstellt. Dabei werden moderne Technologien aus den Bereichen Webentwicklung und Datenbanken eingesetzt, die eine strukturierte Datenverarbeitung und eine flexible Erweiterbarkeit ermöglichen.

Ein wesentlicher Bestandteil der Anwendung ist der Umgang mit unterschiedlichen Daten, wie Benutzerkonten, Informationen zu den Scootern sowie Nutzungs- und Abrechnungsdaten. Diese müssen sicher gespeichert, verarbeitet und jederzeit abrufbar sein. Ebenso sollen die zentralen Prozesse Registrierung, Login, Ausleihe, Rückgabe und Abrechnung, zuverlässig und nachvollziehbar abgebildet werden.

2. Management Summary

Im Rahmen des Projekts wurde eine Webapplikation zur Verwaltung und Nutzung von E-Scootern konzipiert und umgesetzt. Die Einführung der Applikation soll die innerstädtische Mobilität verbessern, indem Nutzerinnen und Nutzer flexibel auf ein digitales Verleihsystem zugreifen können. Gleichzeitig erhalten lokale Verleihfirmen die Möglichkeit, ihre Scooter effizient zu verwalten und bereitzustellen.

Die Lösung basiert auf einer modernen Webarchitektur unter Verwendung der Programmiersprache Python sowie des Web-Frameworks Flask. Für die Speicherung und Verarbeitung der Daten wird ein relationales Datenbanksystem eingesetzt. Die Anwendung bietet eine benutzerfreundliche Weboberfläche für die Interaktion im Browser sowie eine REST-basierte API, über welche ausgewählte Daten auch programmgesteuert abgerufen werden können.

Der zentrale Mehrwert der Applikation liegt in der klar strukturierten Architektur, der einfachen Erweiterbarkeit sowie der Möglichkeit, verschiedene Benutzerrollen und Geschäftsprozesse abzubilden. Funktionen wie Registrierung, Authentifizierung, Scooter-Verwaltung sowie Ausleihe und Abrechnung werden integriert und ermöglichen eine realitätsnahe Anwendung.

Als mögliche Risiken sind insbesondere Aspekte der Sicherheit, der Verfügbarkeit sowie der Skalierbarkeit zu berücksichtigen. So müssen beispielsweise Benutzerdaten geschützt, die Anwendung stabil betrieben und bei steigender Nutzerzahl entsprechend erweitert werden können. Durch den Einsatz bewährter Technologien und einer sauberen Architektur können diese Risiken jedoch gezielt adressiert und reduziert werden.

Das Ergebnis ist eine moderne und effiziente Webapplikation, die die Verwaltung und Ausleihe von E-Scootern wesentlich vereinfacht.

Ausleihunternehmen können ihre gesamte Scooter-Flotte zentral steuern, Fahrzeuge verwalten und wichtige Informationen wie Akkustand und Standort in Echtzeit einsehen. Nutzerinnen und Nutzer profitieren von einer intuitiven Bedienung: Nach der Registrierung lassen sich Scooter bequem ausleihen und wieder zurückgeben, während Fahrtdaten und Kosten automatisch erfasst und abgerechnet werden.

Als Plattform für die Softwareentwicklung wurde GitHub ausgewählt. GitHub stellt eine zuverlässige, skalierbare und weltweit bewährte Infrastruktur für Versionsverwaltung und kollaborative Entwicklung bereit. Dadurch kann das System auch bei wachsender Projektkomplexität effizient weiterentwickelt werden, etwa wenn künftig zusätzliche Fahrzeugtypen wie E-Bikes integriert werden sollen. GitHub unterstützt somit einen flexiblen und nachhaltigen Entwicklungsprozess und bildet eine zukunftssichere Grundlage für den weiteren Ausbau des Mikromobilitätssystems.

3. Zielsetzung

Ziel des Projekts die Konzeption und Entwicklung einer Webapplikation zur Verwaltung und Nutzung eines E-Scooter-Verleihsystems. Die Anwendung soll es Benutzerinnen und Benutzern ermöglichen, sich zu registrieren, einzuloggen sowie verfügbare E-Scooter zu finden, auszuleihen und nach der Nutzung wieder zurückzugeben. Gleichzeitig sollen Verleihfirmen die Möglichkeit erhalten, ihre Scooter-Flotte effizient zu verwalten und deren Status sowie Standort zu aktualisieren.

Im Kontext des Projekts sollen moderne Technologien und Konzepte erfolgreich zur Anwendung kommen. Dazu gehört insbesondere der Einsatz einer relationalen Datenbank zur strukturierten Speicherung der Daten, die Entwicklung der Anwendung mit Python und dem Web-Framework Flask sowie die Umsetzung einer klaren Softwarearchitektur nach dem Model-View-Controller-Prinzip (MVC).

Darüber hinaus soll die Anwendung neben der klassischen Weboberfläche auch eine REST-basierte Schnittstelle (API) bereitstellen, über welche ausgewählten Daten programmgesteuert abgerufen werden können. Dies ermöglicht eine flexible Nutzung der Anwendung und entspricht modernen Anforderungen an Websysteme.

Ein weiterer Schwerpunkt liegt auf der sauberen Planung und Dokumentation der Lösung, für eine vereinfachte Wartung und Nachvollziehbarkeit. Dazu gehören die Modellierung der Datenstruktur, die Beschreibung der Systemarchitektur sowie die Darstellung zentraler Abläufe innerhalb der Anwendung. Es wird angestrebt, eine nachvollziehbare und wartbare Lösung zu entwickeln, welche sich bei Bedarf erweitern lässt, beispielsweise durch zusätzliche Fahrzeugtypen oder neue Funktionen.

Zusammenfassend soll als Endprodukt nicht nur eine funktionierende Webapplikation entstehen, sondern auch ein strukturiertes Gesamtkonzept, das die Anforderungen an eine moderne, skalierbare und erweiterbare Softwarelösung erfüllt.

4. Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben die konkreten Funktionen und Möglichkeiten, welche die Webapplikation den Benutzerinnen und Benutzern zur Verfügung stellt. Sie definieren, was das System leisten muss, unabhängig von der technischen Umsetzung.

Die Webapplikation soll folgende funktionale Anforderungen erfüllen:

1. Benutzerverwaltung

- Benutzerinnen und Benutzer können sich mit einem eindeutigen Benutzernamen sowie einer E-Mail-Adresse registrieren.
- Benutzer können sich mit ihren Zugangsdaten (Benutzername und Passwort) anmelden und abmelden.
- Die Anwendung unterscheidet zwischen verschiedenen Benutzerrollen (z. B. Nutzer, Anbieter, Administrator).
- Benutzer können ihre persönlichen Daten verwalten und bei Bedarf aktualisieren.

2. Scooter-Verwaltung

- Anbieter können neue E-Scooter im System erfassen.
- Jeder Scooter verfügt über eindeutige Attribute wie ID, Standort und Akkustand.
- Anbieter können bestehende Scooter bearbeiten (z. B. Status ändern oder Standort aktualisieren).
- Scooter können bei Bedarf aus dem System entfernt werden.
- Der aktuelle Status eines Scooters (verfügbar, reserviert, in Nutzung) wird im System gespeichert und angezeigt.

3. Ausleihe und Rückgabe

- Benutzer können verfügbare Scooter auswählen und eine Ausleihe starten.
- Der Startzeitpunkt der Nutzung wird automatisch erfasst.
- Während der Nutzung wird der Scooter als „in Verwendung“ markiert.
- Benutzer können die Nutzung beenden und den Scooter zurückgeben.
- Der Endzeitpunkt sowie relevante Nutzungsdaten (z. B. Dauer oder Strecke) werden gespeichert.

4. Abrechnung

- Die Nutzung eines Scooters wird auf Basis einer definierten Preisstruktur berechnet (z. B. Grundpreis und Preis pro Minute).
- Nach Beendigung der Fahrt wird der zu zahlende Betrag automatisch berechnet.
- Die Abrechnung wird im System gespeichert und kann vom Benutzer eingesehen werden.
- Optional können Zahlungsmittel hinterlegt und Transaktionen verarbeitet werden.

5. Weboberfläche

- Die Anwendung stellt eine benutzerfreundliche Weboberfläche bereit, welche über einen Browser zugänglich ist.
- Benutzer können Eingaben tätigen, Aktionen auslösen und Ergebnisse direkt anzeigen lassen.
- Die wichtigsten Funktionen (Login, Scooter-Auswahl, Ausleihe, Rückgabe) sind über die Oberfläche erreichbar.

6. REST API

- Die Anwendung stellt eine REST-basierte API zur Verfügung.
- Über die API können ausgewählte Daten (z. B. Scooter oder Buchungen) abgefragt werden.
- Die API liefert strukturierte Daten im JSON-Format zurück.
- Der Zugriff auf die API erfolgt ohne Browser über geeignete Clients (z. B. Postman oder curl).
- Eine Authentifizierung für den Zugriff auf geschützte Daten ist vorgesehen.

5. Nicht-funktionale Anforderungen

Neben den funktionalen Anforderungen, welche die konkreten Funktionen der Anwendung beschreiben, definieren die nicht-funktionalen Anforderungen die Qualitätsmerkmale des Systems. Sie legen fest, wie gut und unter welchen Bedingungen die Webapplikation ihre Aufgaben erfüllen soll.

Für die vorliegende Webapplikation ergeben sich folgende nicht-funktionale Anforderungen:

1. Sicherheit

Die Sicherheit der Anwendung stellt eine zentrale Anforderung dar, da sensible Benutzerdaten verarbeitet werden.

- Benutzerkonten müssen durch Passwörter geschützt sein.
- Passwörter dürfen nicht im Klartext gespeichert werden, sondern müssen sicher verarbeitet werden (z. B. durch Hashing).
- Der Zugriff auf geschützte Bereiche der Anwendung darf nur nach erfolgreicher Authentifizierung erfolgen.
- Die API muss ebenfalls vor unberechtigtem Zugriff geschützt werden.

2. Performance

Die Anwendung soll eine angemessene Reaktionszeit auf Benutzeranfragen gewährleisten.

- Seiten und Funktionen sollen ohne merkliche Verzögerung geladen werden.
- Die Anwendung soll in der Lage sein, mehrere gleichzeitige Benutzeranfragen zu verarbeiten.
- Datenbankabfragen sollen effizient gestaltet werden, um unnötige Verzögerungen zu vermeiden.

3. Skalierbarkeit

Die Lösung soll so konzipiert sein, dass sie bei steigender Nutzung erweitert werden kann.

- Die Architektur soll es ermöglichen, zusätzliche Benutzer und Datenmengen zu verarbeiten.
- Neue Funktionen oder Erweiterungen (z. B. zusätzliche Fahrzeugtypen wie E-Bikes) sollen ohne grundlegende Änderungen integrierbar sein.

4. Wartbarkeit

Die Anwendung soll einfach verständlich und erweiterbar sein.

- Der Code soll klar strukturiert und nachvollziehbar aufgebaut sein.
- Die Trennung von Verantwortlichkeiten (z. B. durch MVC-Architektur) soll eingehalten werden.
- Änderungen oder Erweiterungen sollen ohne grosse Auswirkungen auf das Gesamtsystem möglich sein.

5. Verfügbarkeit

Die Webapplikation soll zuverlässig und möglichst jederzeit erreichbar sein.

- Die Anwendung muss über das Internet zugänglich sein.
- Ausfälle sollen möglichst vermieden oder schnell behoben werden.
- Während der Prüfungsphase muss die Anwendung stabil verfügbar bleiben.

6. Benutzerfreundlichkeit

Die Anwendung soll einfach und intuitiv bedienbar sein.

- Die Benutzeroberfläche soll klar strukturiert und verständlich sein.
- Wichtige Funktionen sollen leicht auffindbar sein.
- Eingaben sollen überprüft werden, um Fehler zu vermeiden.

6. Lösungsansatz: Wahl der Versionskontrollplattform

Für die Verwaltung des Quellcodes der Webapplikation stand zunächst die Entscheidung zwischen GitHub, GitLab und Bitbucket im Raum. Nach einem sorgfältigen Vergleich der jeweiligen Stärken und Schwächen fiel die Wahl schliesslich auf GitHub als zentrale Plattform für die Versionskontrolle.

Begründung der Wahl:

Wartbarkeit:

- GitHub überzeugt durch eine benutzerfreundliche Oberfläche, ein grosses Ökosystem und zahlreiche Integrationsmöglichkeiten wie CI/CD-Pipelines, Issue-Tracking oder Pull-Requests. Dadurch lassen sich Änderungen im Code klar nachvollziehen, Review-Prozesse effizient durchführen und die langfristige Wartbarkeit des Projekts sicherstellen.

Skalierbarkeit:

- Ob kleines Projekt oder grosses Entwicklungsteam – GitHub wächst flexibel mit. Die Nutzung von Branches, Forks und Pull-Requests ermöglicht eine strukturierte Weiterentwicklung, ohne dass die Stabilität des Hauptcodes gefährdet wird. So bleibt das Projekt auch bei zukünftigen Erweiterungen gut handhabbar.

Verfügbarkeit:

- Als Cloud-basierte Plattform bietet GitHub eine hohe Ausfallsicherheit und ist weltweit erreichbar. Das erleichtert die Zusammenarbeit im Team erheblich, insbesondere wenn Mitglieder remote arbeiten oder von verschiedenen Standorten aus zugreifen.

Vor- und Nachteile im Vergleich zu GitLab und Bitbucket

Kriterium	GitHub	GitLab	Bitbucket
Wartbarkeit	Sehr gute Integrationen, grosse Community	Gute Integrationen, optional selbst hostbar	Einfach, aber weniger verbreitet
Skalierbarkeit	Hoch, ideal für grosse Teams und Projekte	Ebenfalls hoch, starke CI/CD-Funktionen	Mittel, eher für kleinere Teams geeignet
Verfügbarkeit	Global erreichbar, cloud-basiert	Cloud oder self-hosted	Cloud-basiert, Funktionsumfang eingeschränkt
Besonderheiten	Sehr weit verbreitet, viele Tutorials	Mehr Features in der kostenpflichtigen Version	Starke Anbindung ans Atlassian-Ökosystem

Die Entscheidung für GitHub unterstützt eine gut strukturierte und transparente Entwicklung, erleichtert die Wartung des Codes und bietet eine skalierbare, jederzeit verfügbare Plattform. Damit ist GitHub die sinnvollste und zukunftssichere Wahl für dieses Projekt.

7. Abgrenzungen

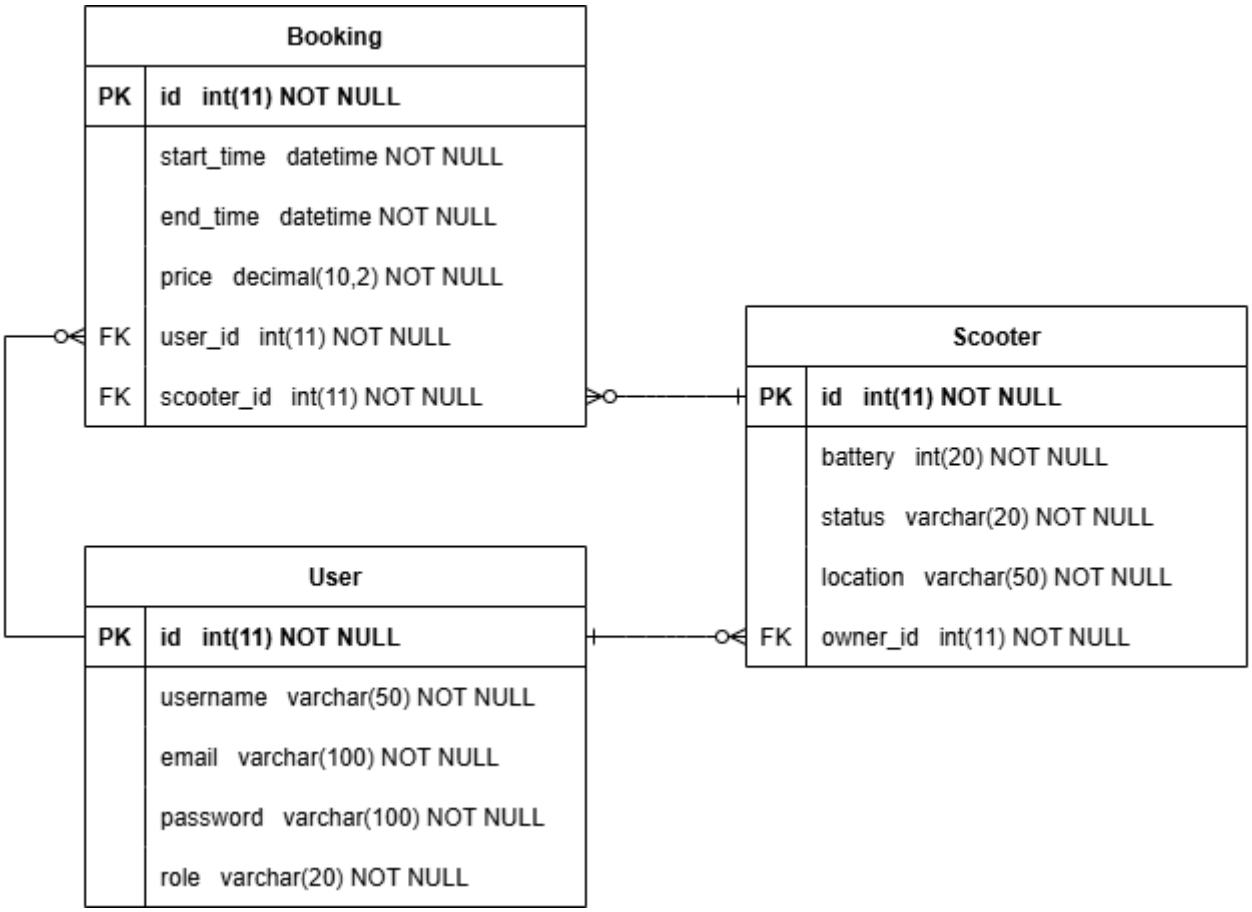
Bezogen auf die Durchführung des Projekts wurde bewusst darauf verzichtet, gewisse Funktionen vollständig umzusetzen, um die Komplexität der Anwendung auf ein sinnvolles und realisierbares Mass zu beschränken. Es wurde angestrebt, eine funktionierende und klar strukturierte Webapplikation zu entwickeln, welche die zentralen Anforderungen erfüllt, ohne unnötig komplex zu werden.

Insbesondere wurde auf die Implementierung einer eigenständigen Zahlungsabwicklung verzichtet. Eine separate Payment-Entität wurde nicht umgesetzt, da die Abrechnung direkt in der Buchung abgebildet wird. Dies reduziert die Komplexität und erhöht die Wartbarkeit, da weniger Abhängigkeiten zwischen den Tabellen bestehen.

Darüber hinaus wurde auf die Integration externer Zahlungsdienstleister sowie auf erweiterte Sicherheitsmechanismen im Zahlungsbereich verzichtet, da diese den geplanten Projektrahmen überschreiten würden. Ebenso wurden keine erweiterten Funktionen wie detaillierte GPS-Tracking-Systeme oder Echtzeit-Kommunikation implementiert.

Die oben genannten Abgrenzungen ermöglichen es, den Fokus auf die Kernfunktionalitäten der Anwendung zu legen, insbesondere auf die Benutzerverwaltung, die Scooter-Verwaltung sowie den Ausleih- und Abrechnungsprozess. Gleichzeitig bleibt die Architektur so gestaltet, dass eine Erweiterung der Anwendung in Zukunft grundsätzlich möglich ist.

8. Datenbankmodell (ERD)



Das Datenmodell besteht aus den drei zentralen Entitäten User, Scooter und Booking. Die Entität User verwaltet die Benutzer der Anwendung, während Scooter die verfügbaren Fahrzeuge repräsentiert. Die Entität Booking stellt die Verbindung zwischen Benutzer und Scooter dar und bildet den Ausleihprozess ab.

Ein Benutzer kann mehrere Buchungen durchführen, während ein Scooter ebenfalls mehrfach ausgeliehen werden kann. Jede Buchung ist jedoch eindeutig einem Benutzer und einem Scooter zugeordnet. Die Struktur des Modells ist normalisiert und vermeidet Redundanzen.

9. Architektur (Model-View-Controller)

Die vorliegende Webapplikation basiert auf dem Model-View-Controller (MVC) Architekturprinzip. Dieses Konzept ermöglicht eine klare Trennung der verschiedenen Verantwortungsbereiche innerhalb der Anwendung und trägt wesentlich zur Wartbarkeit, Erweiterbarkeit und Übersichtlichkeit der Software bei.

8.1 Model

Das Model repräsentiert die Datenstruktur sowie die Geschäftslogik der Anwendung. In der vorliegenden Lösung wird das Model durch die relationale Datenbank sowie die entsprechenden Datenbanktabellen (User, Scooter, Booking) abgebildet.

Das Model ist verantwortlich für:

- die Speicherung und Verwaltung der Daten
- die Definition von Beziehungen zwischen den Entitäten
- die Berechnung von Werten, wie beispielsweise der Preis einer Buchung

Durch die Verwendung eines strukturierten Datenmodells wird sichergestellt, dass Daten konsistent und effizient verarbeitet werden können.

8.2 View

Die View stellt die Benutzeroberfläche der Anwendung dar und ist für die Darstellung der Inhalte im Webbrowser zuständig. Sie umfasst die HTML-Seiten sowie die Darstellung der Daten für den Benutzer.

Die View ist verantwortlich für:

- die Anzeige von Informationen (z. B. verfügbare Scooter)
- die Darstellung von Formularen (z. B. Login, Registrierung)
- die Ausgabe von Ergebnissen und Rückmeldungen

Die View enthält keine Geschäftslogik, sondern dient ausschliesslich der Präsentation der Daten.

8.3 Controller

Der Controller übernimmt die Steuerung der Anwendung und bildet die Schnittstelle zwischen Model und View. In dieser Webapplikation wird der Controller durch die Flask-Routen implementiert.

Der Controller ist verantwortlich für:

- die Verarbeitung von Benutzeranfragen (HTTP-Requests)
- die Steuerung der Abläufe innerhalb der Anwendung
- die Kommunikation mit dem Model (Daten lesen und schreiben)
- die Auswahl und Übergabe von Daten an die View

Der Controller sorgt dafür, dass Eingaben korrekt verarbeitet und die entsprechenden Ergebnisse an den Benutzer zurückgegeben werden.

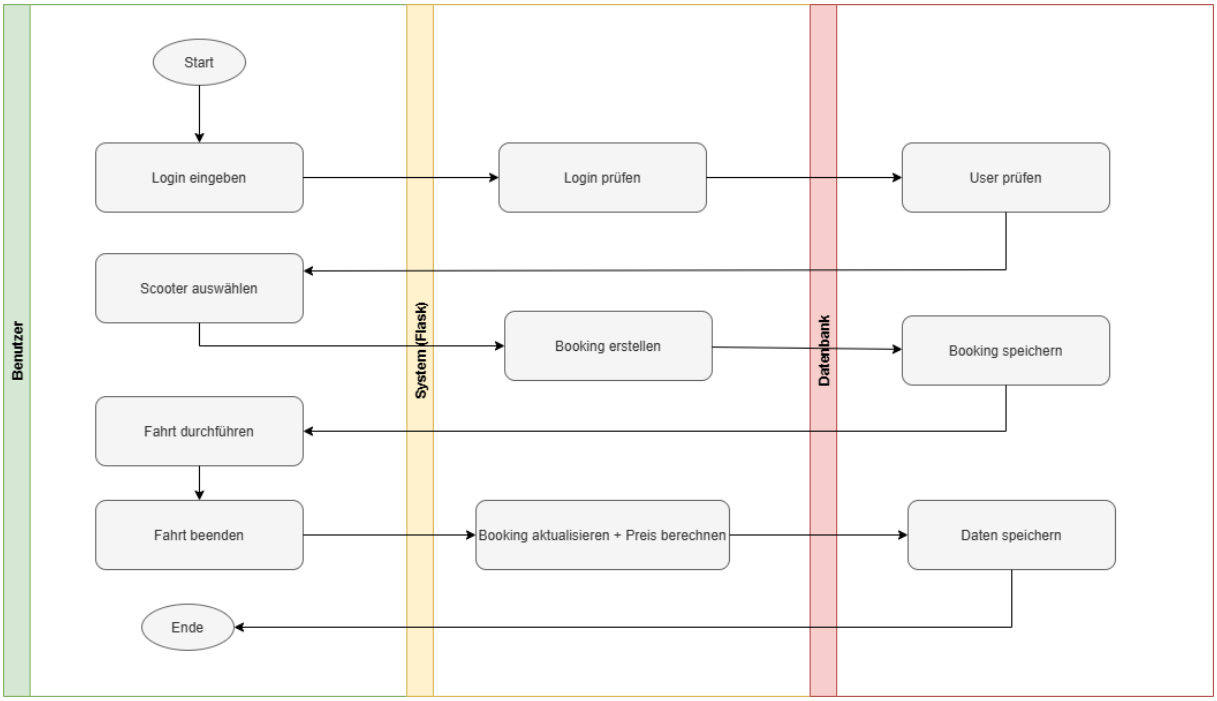
10. Prozess / Workflow

Die zentralen Abläufe der Webapplikation beschreiben die Interaktion zwischen Benutzer, System und Datenbank. Im Folgenden wird der typische Prozess einer Scooter-Ausleihe dargestellt.

Ablauf einer Scooter-Ausleihe:

Der Ausleihprozess besteht aus mehreren aufeinanderfolgenden Schritten:

1. Der Benutzer öffnet die Webapplikation im Browser und meldet sich mit seinen Zugangsdaten an.
2. Nach erfolgreicher Authentifizierung werden dem Benutzer verfügbare Scooter angezeigt.
3. Der Benutzer wählt einen Scooter aus und startet die Ausleihe.
4. Das System erstellt eine neue Buchung (Booking) und speichert den Startzeitpunkt.
5. Während der Nutzung wird der Scooter als „in Verwendung“ markiert.
6. Der Benutzer beendet die Fahrt.
7. Das System speichert den Endzeitpunkt und berechnet den Preis der Nutzung.
8. Die Buchung wird abgeschlossen und dem Benutzer angezeigt.



11. API-Konzept

Neben der Weboberfläche stellt die Anwendung eine REST-basierte API zur Verfügung, über welche ausgewählte Daten programmgesteuert abgerufen werden können. Die API ermöglicht den Zugriff auf zentrale Inhalte der Anwendung ohne Verwendung eines Webbrowsers und kann mit gängigen Tools wie Postman oder curl genutzt werden.

Die API basiert auf dem HTTP-Protokoll und verwendet standardisierte Methoden zur Kommunikation:

- **GET** zum Abrufen von Daten
- **POST** zum Erstellen neuer Daten
- **PUT** bzw. **PATCH** zum Aktualisieren bestehender Daten
- **DELETE** zum Löschen von Daten

Die Daten werden im JSON-Format übertragen, um eine einfache Weiterverarbeitung zu ermöglichen.

Authentifizierung:

Der Zugriff auf geschützte API-Endpunkte erfordert eine Authentifizierung. Diese erfolgt über einen Login-Endpunkt, bei dem Benutzer ihre Zugangsdaten übermitteln.

Nach erfolgreicher Anmeldung erhält der Client eine Bestätigung (z. B. Session oder Token), welche für weitere API-Anfragen verwendet werden kann. Dadurch wird sichergestellt, dass nur berechtigte Benutzer auf die Daten zugreifen können.

API-Endpunkte:

Im Folgenden sind die wichtigsten Endpunkte der API aufgeführt:

Benutzer:

`POST /api/login`

Authentifizierung eines Benutzers

`POST /api/register`

Registrierung eines neuen Benutzers

Scooter:

`GET /api/scooters`

Abrufen aller verfügbaren Scooter

`GET /api/scooters/<id>`

Abrufen eines spezifischen Scooters

Buchungen:

`GET /api/bookings`

Abrufen aller Buchungen eines Benutzers

`POST /api/bookings`

Erstellen einer neuen Buchung

`PUT /api/bookings/<id>`

Aktualisieren einer bestehenden Buchung (z. B. Beenden der Fahrt)

Beispiel für eine Anfrage zum Abrufen aller Scooter:

`GET /api/scooters`

Antwort (JSON):

```
[
  {
    "id": 1,
    "battery": 85,
    "status":
"verfügbar",
    "location": "Bern"
  }
]
```

Nutzen der API:

Die API ermöglicht eine flexible Nutzung der Anwendung, da Daten unabhängig von der Weboberfläche abgerufen und verarbeitet werden können. Dadurch wird die Integration in andere Systeme erleichtert und die Anwendung kann einfacher erweitert werden.

12. Bereitstellung der Anwendung (Deployment)

Für den Betrieb der Webapplikation wird eine klassische Client-Server-Architektur verwendet. Die Anwendung wird über das Internet bereitgestellt, sodass Benutzer über einen Webbrowser darauf zugreifen können. Der Webservice Render wird hier genutzt.

Systemübersicht:

Die Anwendung besteht aus mehreren Komponenten, welche zusammenarbeiten:

- **Client (Browser):**
  Benutzer greifen über einen Webbrowser auf die Anwendung zu und senden HTTP-Anfragen an den Server.

- **Webserver (Flask / Gunicorn):**
  Der Webserver verarbeitet die eingehenden Anfragen, führt die Geschäftslogik aus und liefert die entsprechenden Antworten zurück.

- **Datenbank:**
  Die Datenbank speichert alle relevanten Informationen, wie Benutzer, Scooter und Buchungen.

Ablauf einer Anfrage:

Der typische Ablauf einer Anfrage erfolgt wie folgt:

1. Der Benutzer sendet eine Anfrage über den Webbrowser
2. Der Webserver (Flask) verarbeitet die Anfrage
3. Die Anwendung greift auf die Datenbank zu
4. Die Daten werden verarbeitet und aufbereitet
5. Die Antwort wird an den Benutzer zurückgesendet

Technologische Umsetzung:

Für die Umsetzung werden folgende Technologien eingesetzt:

- **Programmiersprache:** Python
- **Web-Framework:** Flask
- **Webserver:** Gunicorn, Render
- **Datenbank:** Relationales Datenbanksystem (z.B. SQLite)

Optional kann zusätzlich ein Reverse Proxy (z. B. Nginx) verwendet werden, um die Anwendung effizient bereitzustellen.

Verfügbarkeit und Zugriff:

Die Anwendung wird über eine URL im Internet bereitgestellt. Für den Zugriff werden folgende Informationen benötigt:

- URL bzw. IP-Adresse der Anwendung
- Port (falls erforderlich)
- Benutzername und Passwort für den Login

Die Anwendung bleibt während der Prüfungsphase stabil verfügbar, sodass sie durch den Examinator getestet werden kann.

13. Realisierung

12.1 Technischer Aufbau und Grundlagen

In diesem Abschnitt wird die technische Umsetzung der Webapplikation beschrieben. Die Implementierung erfolgt mit Python und dem Web-Framework Flask. Zusätzlich wird eine relationale Datenbank zur Speicherung der Daten verwendet.

Einrichtung der Entwicklungsumgebung

Zu Beginn wird die Entwicklungsumgebung vorbereitet. Die Umsetzung erfolgt mit Python sowie verschiedenen benötigten Bibliotheken.

Installation der benötigten Pakete:

```
$ pip install flask
pip install flask_sqlalchemy
pip install flask_login
pip install flask_migrate
```

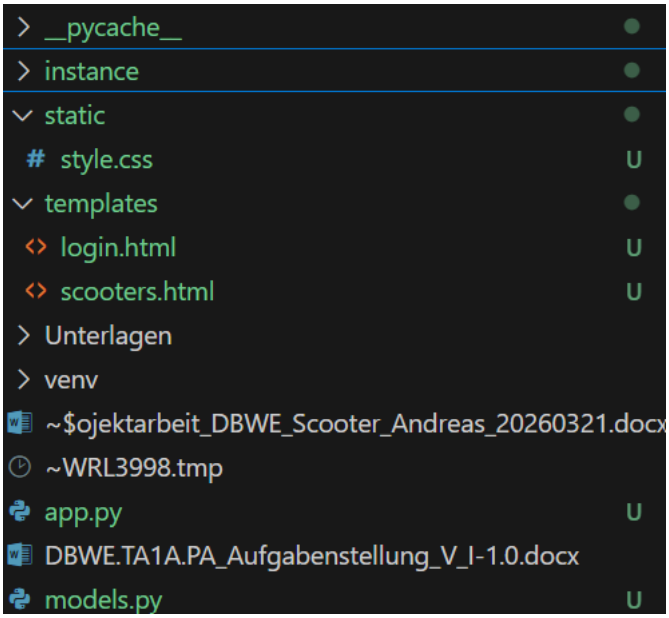
Aktualisieren einer bestehenden Buchung (z. B. Beenden der Fahrt)

Diese Bibliotheken erfüllen folgende Aufgaben:

- Flask: Web-Framework zur Erstellung der Anwendung
- Flask_SQLAlchemy: Verbindung zur Datenbank
- Flask_Login: Verwaltung von Benutzer-Authentifizierung
- Flask_Migrate: Datenbank-Migrationen

Projektstruktur

Die Anwendung wird in einer klar strukturierten Ordnerstruktur aufgebaut:



Diese Struktur ermöglicht eine saubere Trennung der verschiedenen Komponenten der Anwendung.

Erstellung der ersten Flask-Anwendung

Zunächst wird eine einfache Flask-Anwendung erstellt, um die Funktionsfähigkeit der Umgebung zu überprüfen.

```python
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello World"

if __name__ == "__main__":
    app.run(debug=True)
```

Die Anwendung kann mit folgendem Befehl gestartet werden: *python test.py*

Anschliessend ist die Anwendung im Browser unter http://127.0.0.1:5000 erreichbar.

Integration der Datenbank

Die Verbindung zur Datenbank wird mit SQLAlchemy umgesetzt.

```python
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

# DB Konfiguration
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///database.db"
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

db = SQLAlchemy(app)

@app.route("/")
def home():
    return "DB verbunden!"

if __name__ == "__main__":
    app.run(debug=True)
```

Damit wird eine lokale SQLite-Datenbank erstellt, welche die Anwendungsdaten speichert.

Definition der Datenmodelle

Die Datenbanktabellen werden basierend auf dem zuvor definierten ERD umgesetzt.

```
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), nullable=False)
    email = db.Column(db.String(100), nullable=False)
    password = db.Column(db.String(100), nullable=False)
    role = db.Column(db.String(20), nullable=False)
```

Erstellung der Datenbank

Die Datenbank kann mit folgendem Befehl initialisiert werden:

```
>>> from app import app
... from models import db
...
... with app.app_context():
...     db.create_all()
```

Einfügen der Testdaten

Zur Überprüfung der Funktionalität der Datenbank wurden Testdaten eingefügt. Dabei wird ein Benutzer erstellt und in der Datenbank gespeichert. Dieser Schritt dient dazu, die korrekte Verbindung zwischen Anwendung und Datenbank zu validieren.

```
>>> from app import app
>>> from models import db, Scooter, User
>>>
>>> with app.app_context():
...     user = User(username="admin", email="admin@test.ch", password="1234", role="anbieter")
...     db.session.add(user)
...     db.session.commit()
```

12.2 Implementierung der Funktionalitäten

Einfügen von Scootern

Im Rahmen der Implementierung wird die Möglichkeit geschaffen, neue Scooter in der Datenbank zu speichern. Dieser Schritt dient dazu, das zuvor definierte Datenmodell mit konkreten Daten zu befüllen und die Funktionalität der Anwendung zu überprüfen.

Die Umsetzung erfolgt über das ORM-Framework SQLAlchemy, welches eine objektorientierte Interaktion mit der relationalen Datenbank ermöglicht. Dabei werden Instanzen der Klasse Scooter erzeugt und anschliessend in der Datenbank persistiert.

Erstellung der Scooter-Objekte (im Python Terminal):

```
scooter1 = Scooter(
  battery=80,
  status="verfügbar",
  location="Bern",
  owner_id=1
)

scooter2 = Scooter(
  battery=50,
  status="verfügbar",
  location="Zürich",
  owner_id=1
)
```

In diesem Schritt werden zwei Scooter-Objekte erzeugt. Die Attribute entsprechen den im Datenbankmodell definierten Feldern. Der *owner_id*-Wert stellt dabei einen Fremdschlüssel dar, welcher auf einen bestehenden Benutzer verweist.

Speicherung in der Datenbank:

```
db.session.add(scooter1)
db.session.add(scooter2)
db.session.commit()
```

Die erstellten Objekte werden zunächst zur aktuellen Datenbanksession hinzugefügt. Durch den anschliessenden Aufruf von *commit()* wird die Transaktion abgeschlossen und die Daten dauerhaft in der Datenbank gespeichert.

Dieser Vorgang entspricht einer klassischen Schreiboperation in einem relationalen Datenbanksystem.

Bedeutung für die Anwendung

Durch das Einfügen der Scooter-Daten wird die Grundlage für die weiteren Funktionalitäten der Anwendung geschaffen. Insbesondere ermöglicht dieser Schritt die Anzeige von verfügbaren Scootern sowie die Durchführung von Buchungen durch Benutzer.

Anzeige von Scootern

Die Anzeige der verfügbaren Scooter erfolgt über eine dedizierte Route innerhalb der Flask-Anwendung. Dabei werden die in der Datenbank gespeicherten Scooter-Daten ausgelesen und an ein HTML-Template übergeben.

Datenabfrage aus der Datenbank:

```
@app.route("/scooters")
def scooters():
    scooters = Scooter.query.all()
```

```
return render_template(
    "scooters.html",
    scooters=scooters,
    active_booking=active_booking,
    duration=duration
)
```

Mit der Methode *query.all()* werden alle vorhandenen Scooter-Objekte aus der Datenbank geladen. Diese werden anschliessend an das Template *scooters.html* übergeben.

Darstellung im Frontend:

```
<p><strong>Scooter {{ s.id }}</strong></p>
<p>
    Batterie:
    <span class="{{ 'green' if s.battery > 50 else 'red' }}">
        {{ s.battery }}%
    </span>
</p>
<p>Standort: {{ s.location }}</p>
```

Im Template erfolgt die Darstellung mittels einer Schleife über alle Scooter. Die Attribute werden dynamisch eingebunden und im Frontend angezeigt.

Dieser Mechanismus ermöglicht eine klare Trennung zwischen Datenhaltung (Backend) und Darstellung (Frontend), wie sie im MVC-Konzept vorgesehen ist.

Benutzer-Login

Die Authentifizierung erfolgt durch serverseitige Validierung der eingegebenen Benutzerdaten im Rahmen eines HTTP-POST-Requests.

Die eingegebenen Daten werden aus dem Formular gelesen und mit den gespeicherten Werten in der Datenbank verglichen. Wird ein passender Benutzer gefunden, erfolgt die Speicherung der Benutzerinformationen in der Session.

Verwendung der Session:

```
session["user_id"] = user.id
session["username"] = user.username
```

Die Session dient dazu, den eingeloggten Benutzer während der gesamten Nutzung der Anwendung zu identifizieren. Dadurch können nachfolgende Aktionen eindeutig einem Benutzer zugeordnet werden.

Buchung starten

Die Buchung eines Scooters wird durch den Benutzer im Frontend ausgelöst und im Backend verarbeitet. Dabei wird ein neuer Eintrag in der Booking-Tabelle erstellt.

```
@app.route("/start/<int:scooter_id>")
def start_booking(scooter_id):
    if "user_id" not in session:
        return redirect("/login")

    booking = Booking(
        user_id=session["user_id"],
        scooter_id=scooter_id,
        start_time=datetime.now()
    )
```

Beim Start einer Fahrt wird ein neues *Booking*-Objekt erzeugt. Die Attribute umfassen den Benutzer, den gewählten Scooter sowie den Startzeitpunkt.

Speicherung der Buchung:

```
db.session.add(booking)
db.session.commit()
```

Das Objekt wird der Datenbanksession hinzugefügt und anschliessend persistent gespeichert. Damit wird die Buchung dauerhaft in der Datenbank abgelegt.

Buchung beenden und Preisberechnung

Beim Beenden einer Fahrt wird die bestehende Buchung aktualisiert. Zusätzlich erfolgt die Berechnung des Preises basierend auf der Fahrtdauer.

```
@app.route("/stop/<int:booking_id>")
def stop_booking(booking_id):
    booking = Booking.query.get(booking_id)

    booking.end_time = datetime.now()
```

Zunächst wird die entsprechende Buchung anhand ihrer ID geladen und der Endzeitpunkt gesetzt.

Berechnung der Fahrtdauer:

```
duration = (booking.end_time - booking.start_time).total_seconds() / 60
```

Die Dauer der Fahrt wird in Minuten berechnet. Dazu wird die Differenz zwischen Start- und Endzeit bestimmt.

Preisberechnung:

```
booking.price = round(duration * 0.2, 2)
```

Der Preis wird dynamisch auf Basis der Fahrtdauer berechnet. In diesem Fall wird ein Tarif von 0.20 CHF pro Minute verwendet. Das Ergebnis wird auf zwei Dezimalstellen gerundet.

Speicherung der Änderungen:

```
db.session.commit()
```

Durch den Commit werden die Änderungen dauerhaft in der Datenbank gespeichert. Dieser Schritt entspricht einer Update-Operation im relationalen Datenbanksystem.

14. Testprotokoll

Im folgenden Testprotokoll werden die durchgeführten Tests zur Überprüfung der Funktionalität der Webapplikation dokumentiert. Jeder Testfall enthält eine Kurzbeschreibung, das erwartete Ergebnis sowie das tatsächlich erzielte Resultat.

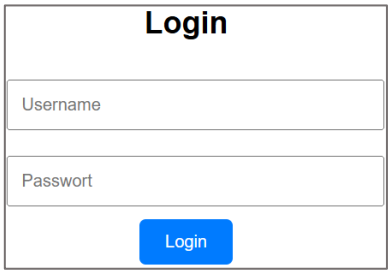
Testfall-Nr.	Datum	Tester	Beschreibung	Erwartetes Ergebnis	Tatsächliches Ergebnis
1	10.02.2026	A. Geiger	Benutzer-Login mit korrekten Daten	Benutzer wird erfolgreich eingeloggt und auf die Scooter-Übersicht weitergeleitet	Erfüllt
2	10.02.2026	A. Geiger	Benutzer-Login mit falschen Zugangsdaten	Fehlermeldung wird angezeigt, Login wird verweigert	Erfüllt
3	17.02.2026	A. Geiger	Anzeige der Scooter-Liste	Alle in der Datenbank gespeicherten Scooter werden korrekt angezeigt	Erfüllt
4	16.03.2026	A. Geiger	Start einer Fahrt (Booking)	Neue Buchung wird erstellt und Startzeit gespeichert	Erfüllt
5	16.03.2026	A. Geiger	Beenden einer Fahrt (Booking)	Endzeit wird gesetzt und Preis korrekt berechnet	Erfüllt
6	20.03.2026	A. Geiger	Anzeige der aktuellen Fahrt	Laufende Fahrt wird angezeigt inkl. Dauer und Stop-Button	Erfüllt
7	20.03.2026	A. Geiger	Anzeige des letzten Preises	Nach Beenden der Fahrt wird der letzte Preis korrekt angezeigt	Erfüllt

15. User Manual

Die Webanwendung wird über einen Browser aufgerufen und ermöglicht die Verwaltung und Nutzung von Scootern.

Login

Nach dem Aufruf der Anwendung erscheint eine Login-Seite. Der Benutzer gibt seinen Benutzernamen und sein Passwort ein, um sich anzumelden.



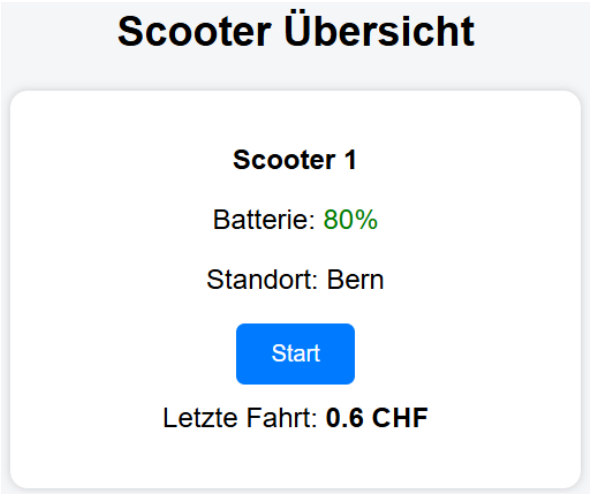
Anzeige der Scooter

Nach erfolgreichem Login wird eine Übersicht aller verfügbaren Scooter angezeigt.
Für jeden Scooter werden der Batteriestand sowie der Standort dargestellt.

Fahrt starten und beenden

Durch Klick auf den „Start“-Button wird eine Fahrt begonnen. Dabei wird automatisch eine Buchung erstellt und der Startzeitpunkt gespeichert.

Mit dem „Stop“-Button kann die laufende Fahrt beendet werden. Dabei wird die Dauer berechnet und ein Preis ermittelt.



16. Fazit

Insgesamt verdeutlicht die Entwicklung dieser Webapplikation, wie sinnvoll digitalisierte Prozesse die Verwaltung und Nutzung von E-Scootern vereinfachen können. Durch die zentrale Erfassung aller Abläufe reduziert sich der Verwaltungsaufwand spürbar, während gleichzeitig präzise Echtzeitdaten für eine gezielte Mobilitätsplanung zur Verfügung stehen. Auch für die Nutzer: innen entsteht ein spürbarer Mehrwert durch eine einfache Ausleihe und automatische Abrechnung, während lokale Verleihfirmen ihre Flotten komfortabel und effizient überwachen können.

Die skalierbare und erweiterbare Architektur sorgt dafür, dass die Lösung nicht nur den aktuellen Anforderungen entspricht, sondern auch problemlos um zukünftige Mobilitätsangebote erweitert werden kann. Technische Grundlagen wie eine klar strukturierte Datenbank, das Model-View-Controller-Prinzip und durchdachte Sicherheitsmassnahmen tragen zu einem stabilen, wartbaren und verlässlichen Betrieb bei.

Ganz bewusst wurde auf besonders komplexe Funktionen wie externe Bezahlsysteme oder ein detailliertes GPS-Tracking verzichtet, um den Fokus auf die wesentlichen Kernfunktionen zu legen und die Anwendung übersichtlich zu halten. Diese klare Abgrenzung schafft Transparenz und lässt gleichzeitig genügend Raum für spätere Erweiterungen.

So entsteht eine robuste und zukunftsfähige Basis für eine moderne, datengetriebene und effizient organisierte urbane Mobilität, die Stadtverwaltung, Ausleihunternehmen und Nutzer gleichermassen unterstützt.

17. Reflexion

Das Projekt zur Entwicklung einer Webapplikation für ein E-Scooter-Verleihsystem verlief sehr gut und ohne grössere Schwierigkeiten. Von der ersten Anforderungsanalyse bis hin zur fertigen Anwendung konnten alle geplanten Arbeitsschritte erfolgreich umgesetzt werden.

Besonders positiv ist mir die klare Struktur des Projekts aufgefallen. Die Unterteilung in die Phasen Analyse, Design, Implementierung und Test hat für einen übersichtlichen und zielgerichteten Ablauf gesorgt. Auch die Entscheidung für moderne Technologien wie Python, Flask und eine relationale Datenbank hat sich als absolut sinnvoll erwiesen, da sie eine flexible und effiziente Umsetzung ermöglichten.

Ein wichtiger Lerngewinn war ausserdem die konsequente Anwendung des Model-View-Controller-Prinzips. Die klare Trennung zwischen Datenmodell, Benutzer-oberfläche und Steuerung hat es erleichtert, einzelne Bestandteile unabhängig voneinander zu entwickeln und zu testen.

Sowohl die funktionalen als auch die nicht-funktionalen Anforderungen konnten vollständig erfüllt werden. Dazu gehören etwa die Benutzer- und Scooter-Verwaltung, Ausleihe und Rückgabe, die Abrechnung sowie die Entwicklung einer REST-API. Selbst bei simulierten Mehrfachzugriffen lief die Anwendung stabil und performant, sodass alle angestrebten Qualitätsziele erreicht wurden.

Abschliessend lässt sich sagen, dass das Projekt nicht nur eine funktionierende Web-applikation hervorgebracht hat, sondern auch wesentlich zur Weiterentwicklung meiner Fähigkeiten in den Bereichen Softwareentwicklung, Datenmodellierung und Projekt-organisation beigetragen hat. Die Arbeit bietet eine solide Basis für zukünftige Erweiterungen, wie etwa die Einbindung weiterer Fahrzeugtypen oder die Integration externer Bezahlsysteme, und kann damit problemlos weiter ausgebaut werden.

18. Quellenverzeichnis

Atlassian Dokumentation. (2026). https://support.atlassian.com. *Internetrecherche*.

Bitbucket. (2026). https://bitbucket.org. *Internetrecherche*.

Github. (2026). https://github.com. *Internetrecherche*.

GitLab. (2026). https://docs.gitlab.com. *Internetrecherche*.

GitLab Feature Comparison. (2026). https://about.gitlab.com/features/. *Internetrecherche*.

19. Abkürzungsverzeichnis

Abkürzung	**Bedeutung**
E-Scooter	Elektroroller (Elektro-Scooter)
MVC	Model-View-Controller (Softwarearchitektur)
API	Application Programming Interface (Programmierschnittstelle)
JSON	JavaScript Object Notation (Datenformat)
HTTP	Hypertext Transfer Protocol (Kommunikationsprotokoll)
REST	Representational State Transfer (Architekturstil für Webservices)
ERD	Entity-Relationship-Diagramm

20. Glossar

Begriff	Erklärung
Webapplikation	Software, die über einen Webbrowser zugänglich ist
Benutzerrollen	Unterschiedliche Rechte und Funktionen für Nutzer, Anbieter und Administrator
Datenbank	Strukturierte Sammlung von Daten, hier relationale Datenbank
REST API	Schnittstelle für den Datenzugriff durch andere Programme
Sicherheitsmassnahmen	Massnahmen zum Schutz von Benutzerdaten (z. B. Passwort-Hashing)
Skalierbarkeit	Erweiterbarkeit des Systems, bei steigender Nutzung
Performance	Geschwindigkeit und Reaktionsfähigkeit der Anwendung
Model-View-Controller (MVC)	Softwarearchitektur, die Daten, Darstellung und Steuerung trennt
Echtzeitdaten	Daten, die sofort verarbeitet und angezeigt werden
Entity-Relationship-Diagramm (ERD)	Diagramm, das die Datenstruktur einer Datenbank visuell darstellt